



-

UNIVERSIDAD DON BOSCO

DESARROLLO MOVILES

FORO 2

Presenta:

Oscar Alexander Quintanilla Rodríguez - QR200363

Oscar Dave Guerra Pacheco - GP200006

Docente:

Alexander Siguenza

Fecha:

11 de Mayo de 2026

Índice

Índice.....	2
Introducción	3
Investigación sobre Autenticación en Firebase.....	4
Definición y Funcionamiento General de Firebase Authentication.....	4
Tipos de Autenticación Disponibles	4
Correo Electrónico y Contraseña	5
Inicio de Sesión con Google	5
Casos de Uso Típicos según el Tipo de Aplicación.....	6
Consideraciones de Seguridad	7
Documentación de Implementación de Autenticación por Correo Electrónico	8
Fase 1: Configuración del Proyecto en Firebase Console	8
Fase 2: Integración del SDK en Android Studio	9
Fase 3: Habilitación del Proveedor de Correo Electrónico.....	9
Fase 4: Implementación del Flujo de Login	9
Fase 5: Validaciones y Manejo de Errores	10
Desarrollo Técnico de la Aplicación.....	11
Arquitectura General.....	11
Estructura de Datos en Cloud Firestore	11
Modelos de Datos	12
Capa de Datos: Repositorios	13
Capa de Presentación: ViewModels	13
Capa de Interfaz de Usuario: Pantallas en Jetpack Compose	13
Funcionalidades Implementadas.....	13
Decisiones Técnicas Notables.....	14
Capturas de Pantalla del Proyecto en Ejecución.....	15
Flujo de Autenticación.....	15
Verificación en la Consola de Firebase	20
Conclusiones	21
Referencias.....	22

Introducción

La gestión de las finanzas personales se ha convertido en una competencia indispensable en el contexto económico actual. La proliferación de plataformas digitales de pago, la diversificación de fuentes de ingreso y la velocidad con la que se realizan transacciones cotidianas han generado la necesidad de contar con herramientas tecnológicas que permitan a las personas registrar, organizar y analizar sus gastos de manera eficiente. En este sentido, las aplicaciones móviles representan una solución natural, ya que el teléfono inteligente acompaña al usuario durante todo el día y permite capturar información en el momento exacto en que ocurre la transacción.

El presente trabajo documenta el diseño y desarrollo de una aplicación móvil para Android, implementada en el lenguaje de programación Kotlin con el entorno de desarrollo Android Studio, cuyo objetivo es permitir a los usuarios registrar y visualizar sus gastos personales. La aplicación integra dos servicios fundamentales de la plataforma Firebase de Google: Firebase Authentication para la gestión de usuarios y Cloud Firestore para el almacenamiento persistente de los datos en la nube. La elección de Firebase responde a su solidez como Backend-as-a-Service (BaaS), su integración nativa con Android y la simplificación de tareas tradicionalmente complejas como la autenticación segura y la sincronización en tiempo real.

La arquitectura de la aplicación se diseñó siguiendo el patrón Modelo-Vista-VistaModelo (MVVM), recomendado oficialmente por Google para el desarrollo de aplicaciones Android modernas. La interfaz de usuario se construyó con Jetpack Compose, el toolkit declarativo de UI que reemplaza progresivamente al sistema tradicional basado en archivos XML. Esta

Investigación sobre Autenticación en Firebase

La autenticación es uno de los componentes críticos de cualquier aplicación moderna que maneje datos personales o que ofrezca experiencias diferenciadas por usuario. Implementar un sistema de autenticación seguro y robusto desde cero requiere conocimientos especializados en criptografía, gestión de tokens, almacenamiento seguro de contraseñas, prevención de ataques de fuerza bruta, recuperación de cuentas y cumplimiento normativo. Por esta razón, la mayoría de los desarrolladores prefieren delegar estas responsabilidades en servicios especializados como Firebase Authentication.

Definición y Funcionamiento General de Firebase Authentication

Firebase Authentication es un servicio gestionado de la plataforma Firebase que provee soluciones listas para la autenticación de usuarios en aplicaciones móviles, web y de servidor. El servicio expone un Software Development Kit (SDK) para múltiples plataformas que se encarga de toda la complejidad relacionada con el registro, inicio de sesión, mantenimiento de la sesión, recuperación de contraseñas y revocación de credenciales (Google, 2024).

Internamente, Firebase Authentication se basa en el estándar JSON Web Token (JWT). Cuando un usuario se autentica con éxito, Firebase emite un ID Token firmado criptográficamente que contiene información del usuario y una fecha de expiración corta (una hora por defecto). Este token se renueva de manera transparente mediante un Refresh Token de larga duración, lo que permite mantener sesiones persistentes sin comprometer la seguridad. El SDK también almacena estos tokens de forma segura en el dispositivo, utilizando los mecanismos nativos de cada plataforma.

Tipos de Autenticación Disponibles

Firebase Authentication admite múltiples métodos o proveedores de autenticación, los cuales pueden combinarse en una misma aplicación. A continuación se describen los principales.

Correo Electrónico y Contraseña

Es el método tradicional y el más utilizado en aplicaciones genéricas. El usuario se registra proporcionando una dirección de correo electrónico y una contraseña, y Firebase se encarga de almacenar la contraseña de forma segura mediante funciones de derivación de claves resistentes (Firebase no almacena nunca la contraseña en texto plano). El servicio también ofrece funcionalidades complementarias como envío de correos de verificación de direcciones de correo, restablecimiento de contraseña y cambio de correo asociado a la cuenta.

La principal ventaja de este método es que no depende de ninguna cuenta externa: cualquier persona con un correo válido puede registrarse. Como contrapartida, requiere que el usuario recuerde una contraseña adicional y obliga al desarrollador a implementar validaciones razonables de complejidad.

Inicio de Sesión con Google

Permite al usuario autenticarse mediante su cuenta de Google. El flujo aprovecha el estándar OAuth 2.0 y, en dispositivos Android, suele integrarse a través de la API moderna Credential Manager o, en versiones más antiguas, mediante Google Sign-In de Google Play Services. El usuario selecciona una de las cuentas de Google ya configuradas en el dispositivo y, con un solo toque, queda autenticado.

Sus principales ventajas son la rapidez del registro (no se requiere completar formularios), la confianza que el usuario deposita en la marca Google y la disponibilidad casi universal de cuentas de Google en dispositivos Android. Como limitación, requiere configurar la

huella digital SHA-1 del certificado de la aplicación en la consola de Firebase y depende de los Servicios de Google Play, que pueden no estar disponibles en algunos dispositivos.

Tabla 1

Método	Ventajas	Desventajas	Caso de Uso
Email/contraseña	Universal, sin dependencias externas	Fricción de registro, gestión de contraseñas	Apps generales
Google	Registro instantáneo, alta confianza	Requiere SHA-1 y Play Services	Apps Android masivas
Apple	Privacidad reforzada (oculta el correo)	Configuración compleja en cuentas Apple Developer	Apps multiplataforma con iOS
Redes sociales	Conveniencia para usuarios activos en la red	Dependencia del proveedor; políticas cambiantes	Apps comunitarias
Teléfono (SMS)	Identificador verificable; baja fricción	Costo de SMS; cuota limitada	Mensajería, banca
Anónima	Cero fricción; permite probar la app	Datos volátiles, atados al dispositivo	Onboarding gradual
Token personalizado	Integración con sistemas legacy	Requiere infraestructura de servidor propia	Apps corporativas

Casos de Uso Típicos según el Tipo de Aplicación

La elección del método de autenticación responde más a la naturaleza de la aplicación que a una jerarquía absoluta. Las aplicaciones de finanzas personales como la desarrollada en este proyecto se benefician de combinar correo electrónico (para usuarios que prefieren no asociar su cuenta a redes sociales) con Google Sign-In (para reducir la fricción de registro). Las aplicaciones empresariales suelen optar por Microsoft o por tokens personalizados que se integren con su Active Directory. Las aplicaciones sociales o de comunidades favorecen el inicio

con Facebook o X. Los servicios financieros y de mensajería instantánea privilegian la autenticación por número de teléfono debido a la unicidad y verificabilidad del identificador.

Consideraciones de Seguridad

Aunque Firebase Authentication abstrae gran parte de la complejidad criptográfica, el desarrollador sigue siendo responsable de aplicar buenas prácticas. Las contraseñas deben cumplir un mínimo de complejidad razonable; Firebase exige al menos seis caracteres pero se recomiendan ocho o más. La verificación del correo electrónico se sugiere antes de permitir operaciones sensibles, ya que reduce la creación de cuentas falsas. Los tokens de identidad nunca deben almacenarse en lugares accesibles a otras aplicaciones.

Adicionalmente, las reglas de seguridad de Cloud Firestore deben coordinarse con la autenticación. Sin reglas adecuadas, cualquier usuario autenticado podría leer o escribir documentos pertenecientes a otros usuarios. La práctica recomendada consiste en validar en cada operación que el campo `userId` del documento coincida con el identificador único (UID) del usuario que realiza la solicitud. Esta verificación se realiza en el servidor, no en el cliente, por lo que no puede ser saltada manipulando la aplicación móvil (Firebase, 2024a).

Documentación de Implementación de Autenticación por Correo Electrónico

Esta sección describe paso a paso el procedimiento que se siguió para integrar Firebase Authentication mediante correo electrónico y contraseña en la aplicación de control de gastos. El procedimiento se divide en cinco fases: configuración del proyecto en la consola de Firebase, integración del SDK en Android Studio, habilitación del proveedor de correo electrónico, implementación del flujo de inicio de sesión y, finalmente, validaciones y manejo de errores.

Fase 1: Configuración del Proyecto en Firebase Console

El primer paso consistió en crear un proyecto en la consola web de Firebase, accesible desde la dirección `console.firebase.google.com`. Una vez iniciada sesión con una cuenta de Google, se seleccionó la opción Crear un proyecto y se asignó el nombre Foro2DMS. Durante el asistente de creación se desactivó Google Analytics, ya que no se requiere para el alcance del proyecto académico.

Tras la creación del proyecto, se procedió a registrar la aplicación Android. En la pantalla principal del proyecto se eligió el ícono correspondiente a Android, lo cual abre un formulario donde se debe ingresar el nombre del paquete de la aplicación. Este nombre debe coincidir exactamente con el `applicationId` configurado en el archivo `build.gradle.kts` del módulo `app` del proyecto Android Studio. En el caso de este proyecto, dicho identificador es `com.example.foro2dms`.

Como resultado del registro se obtiene un archivo de configuración llamado `google-services.json`, el cual contiene los identificadores únicos del proyecto Firebase y debe colocarse dentro de la carpeta `app` del proyecto Android Studio. Este archivo es leído automáticamente por el plugin de Gradle de Google Services durante la compilación, lo que evita tener que codificar manualmente claves o identificadores en el código fuente.

[

Fase 2: Integración del SDK en Android Studio

La integración del SDK de Firebase en el proyecto Android Studio se realizó mediante el catálogo de versiones `libs.versions.toml`, una práctica recomendada por Gradle para centralizar la gestión de dependencias. En dicho archivo se agregaron las versiones del Bill of Materials (BOM) de Firebase y del plugin `google-services`, así como las referencias a las librerías de autenticación y Firestore.

Fase 3: Habilitación del Proveedor de Correo Electrónico

Una vez integrado el SDK, fue necesario habilitar el proveedor de correo electrónico en la consola de Firebase. Para ello, dentro del proyecto Foro2DMS se accedió a la sección Authentication y, dentro de ella, a la pestaña Sign-in method. Se seleccionó el proveedor Correo electrónico/contraseña y se activó únicamente la opción principal, dejando desactivada la opción de vínculo por correo (`passwordless email link`), ya que su flujo es diferente y no se contempla en este proyecto.

Fase 4: Implementación del Flujo de Login

Para implementar el flujo de inicio de sesión se siguió el patrón Repository, encapsulando todas las llamadas a Firebase Authentication en una única clase llamada `AuthRepository`. Esta separación de responsabilidades evita que la lógica de la interfaz de usuario o de los ViewModels conozca los detalles del SDK de Firebase, lo que facilita el mantenimiento y las pruebas.

Fase 5: Validaciones y Manejo de Errores

Las validaciones se aplican en dos niveles. En la interfaz de usuario, los botones Ingresar y Crear cuenta permanecen deshabilitados mientras los campos de correo o contraseña estén vacíos, lo que previene clics accidentales. En el ViewModel, se realiza una validación de fondo más estricta antes de invocar al repositorio.

El manejo de errores aprovecha la jerarquía de excepciones de FirebaseAuth para producir mensajes específicos. Por ejemplo, FirebaseAuthInvalidUserException se traduce en "No existe una cuenta con ese correo", FirebaseAuthInvalidCredentialsException en "Credenciales incorrectas", FirebaseAuthUserCollisionException en "Ya existe una cuenta con ese correo" y FirebaseAuthWeakPasswordException en "La contraseña es muy débil". Esta traducción se centraliza en una función mapErrorToMessage del ViewModel, lo que facilita su modificación o internacionalización futura.

Adicionalmente, la sesión del usuario se persiste automáticamente gracias a Firebase Authentication, que almacena los tokens en el almacenamiento privado de la aplicación. Esto significa que si el usuario cierra la aplicación y la vuelve a abrir, no necesita iniciar sesión nuevamente: el LoginViewModel detecta la sesión activa en su bloque init y emite isAuthenticated igual a verdadero, llevándolo directamente a la pantalla principal de gastos.

Desarrollo Técnico de la Aplicación

Esta sección presenta una visión integral del desarrollo de la aplicación, abarcando su arquitectura, modelos de datos, capas de repositorios, ViewModels y pantallas. Aunque la documentación de la implementación de la autenticación por correo se trató en el capítulo anterior por exigencia de la rúbrica, aquí se profundiza en el resto de las funcionalidades: gestión de gastos, filtrado por mes, categorías personalizadas e inicio de sesión con Google.

Arquitectura General

La aplicación sigue la arquitectura Modelo-Vista-VistaModelo (MVVM), recomendada por Google para Android moderno. En MVVM la responsabilidad se distribuye en tres capas: la capa de datos contiene los modelos y los repositorios que acceden a las fuentes de datos (en este caso, Firebase); la capa de presentación contiene los ViewModels, que mantienen el estado de cada pantalla y orquestan las operaciones; finalmente, la capa de UI está compuesta por funciones componibles (Composables) de Jetpack Compose, que se limitan a renderizar el estado y disparar acciones.

Esta separación facilita varios beneficios. En primer lugar, los ViewModels sobreviven a cambios de configuración como rotaciones de pantalla, lo que evita perder el estado del usuario. En segundo lugar, los Composables se mantienen libres de lógica de negocio, lo que los hace fácilmente reutilizables y testeables mediante el sistema de Preview de Compose. En tercer lugar, la capa de datos puede ser sustituida (por ejemplo, reemplazando Firebase por una API REST propia) sin alterar las capas superiores, lo que respeta el principio de inversión de dependencias.

Estructura de Datos en Cloud Firestore

Cloud Firestore es una base de datos NoSQL orientada a documentos, organizada en colecciones jerárquicas. Para esta aplicación se diseñó la siguiente estructura:

```
users/
  {uid}/                                ← documento por usuario
    gastos/                             ← subcolección de gastos
      {gastoId}                         ← documento de gasto
        nombre, monto,
        categoria, fecha,
        userId
    categorias/                         ← subcolección de categorías
      {catId}
        nombre
```

Modelos de Datos

Los datos del dominio se representan mediante data classes de Kotlin, una característica del lenguaje que genera automáticamente los métodos equals, hashCode, toString y copy para clases destinadas exclusivamente a contener datos. Esto reduce el código repetitivo y produce objetos inmutables seguros.

La clase Gasto representa un gasto individual con seis propiedades: identificador, nombre descriptivo, monto numérico, categoría, fecha (almacenada como milisegundos desde la época Unix) e identificador del usuario propietario. La clase Categoria representa una categoría personalizable con identificador y nombre. Es fundamental que todas las propiedades tengan valores por defecto, ya que Firestore exige un constructor sin parámetros para deserializar documentos automáticamente. Sin esta convención, la aplicación lanzaría excepciones en tiempo de ejecución al leer documentos desde la nube.

```
data class Gasto(
    val id: String = "",
    val nombre: String = "",
    val monto: Double = 0.0,
    val categoria: String = "",
    val fecha: Long = System.currentTimeMillis(),
    val userId: String = ""
)
```

Capa de Datos: Repositorios

Se implementaron tres repositorios. AuthRepository encapsula las operaciones de autenticación (descrito en el capítulo anterior). GastoRepository encapsula las operaciones de Firestore para gastos: observación en tiempo real mediante un Flow, inserción y eliminación. CategoriaRepository hace lo análogo para categorías.

Capa de Presentación: ViewModels

Cada pantalla cuenta con su propio ViewModel. LoginViewModel gestiona el estado del inicio de sesión y delega en AuthRepository. GastosViewModel es el más complejo: además de mantener la lista de gastos y categorías, gestiona el mes seleccionado para el filtrado, calcula el total de gastos del mes y combina las categorías predeterminadas del sistema con las personalizadas del usuario.

Capa de Interfaz de Usuario: Pantallas en Jetpack Compose

Las pantallas se construyeron con Jetpack Compose, el toolkit declarativo de UI de Android. A diferencia del sistema clásico basado en archivos XML, Compose permite describir la UI mediante funciones de Kotlin marcadas con la anotación @Composable. Cada función emite los elementos visuales que conforman la pantalla y reacciona automáticamente a los cambios de estado, lo que se conoce como recomposición.

Funcionalidades Implementadas

La aplicación cumple con todos los requisitos funcionales planteados por la rúbrica y agrega varias mejoras opcionales. Entre las funcionalidades incluidas se destacan: el inicio de

sesión con correo y contraseña; el inicio de sesión con Google mediante la API moderna Credential Manager; el registro de gastos con nombre, monto, categoría y fecha; la persistencia en Firestore asociada al usuario autenticado; la visualización del historial de gastos agrupados por día; el cálculo y visualización del total mensual; el filtrado por mes mediante un navegador con flechas; y la gestión de categorías personalizadas adicionales a las predeterminadas.

Decisiones Técnicas Notables

Durante el desarrollo se tomaron varias decisiones técnicas que merecen ser destacadas por su valor pedagógico y por contribuir a la calidad del código entregado.

La primera fue la adopción de la API Credential Manager para el inicio de sesión con Google, en lugar de la API más antigua Google Sign-In de Google Play Services. La API antigua fue marcada como obsoleta por Google en 2023 y, aunque sigue funcionando, no es la solución oficial recomendada para nuevas aplicaciones. Credential Manager unifica el manejo de credenciales y permitirá, en el futuro, integrar con facilidad mecanismos como Passkeys.

Capturas de Pantalla del Proyecto en Ejecución

A continuación se presentan capturas de pantalla del proyecto en ejecución, organizadas por flujo funcional. Cada captura está acompañada de una breve descripción del estado mostrado y de la operación que se está realizando.

Flujo de Autenticación

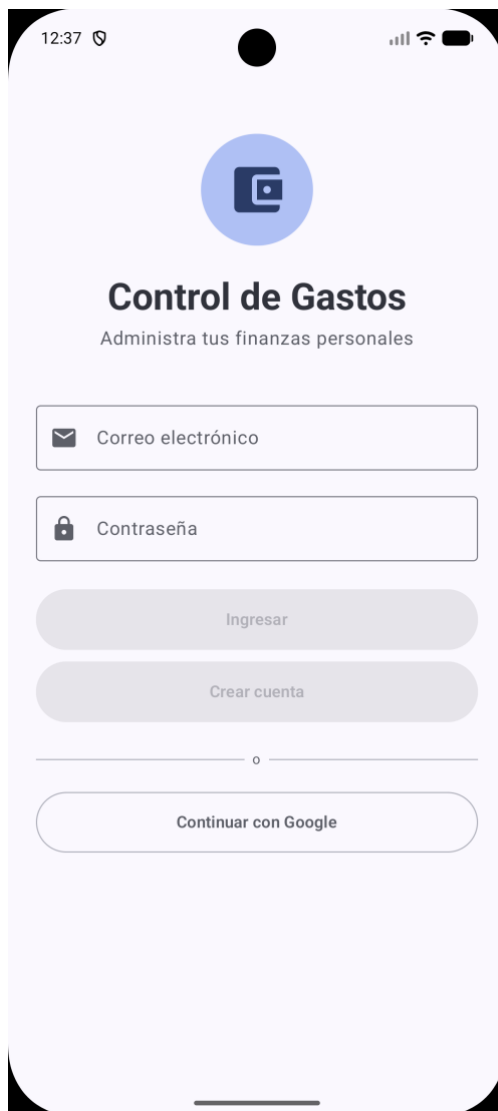


Figura 1. Pantalla inicial de Login. El usuario ve los campos de correo electrónico y contraseña, los botones de Ingresar y Crear cuenta, y la opción de continuar con Google.

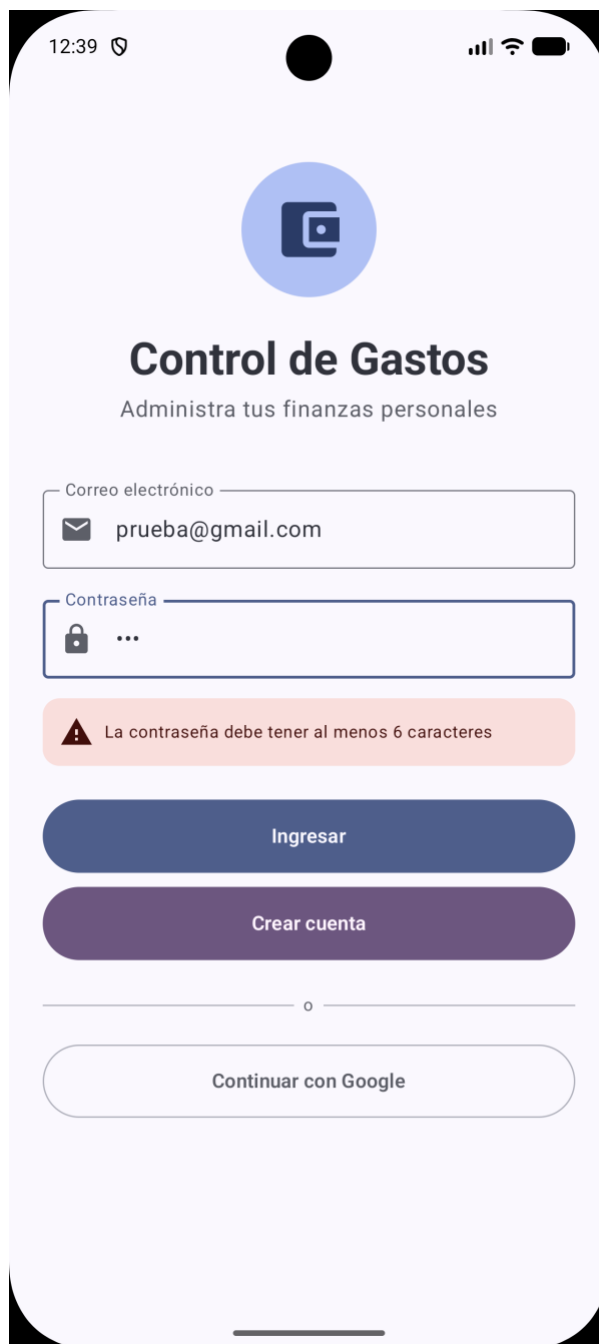


Figura 2. Validación de credenciales. La aplicación traduce los errores técnicos del SDK de Firebase en mensajes legibles para el usuario en español.

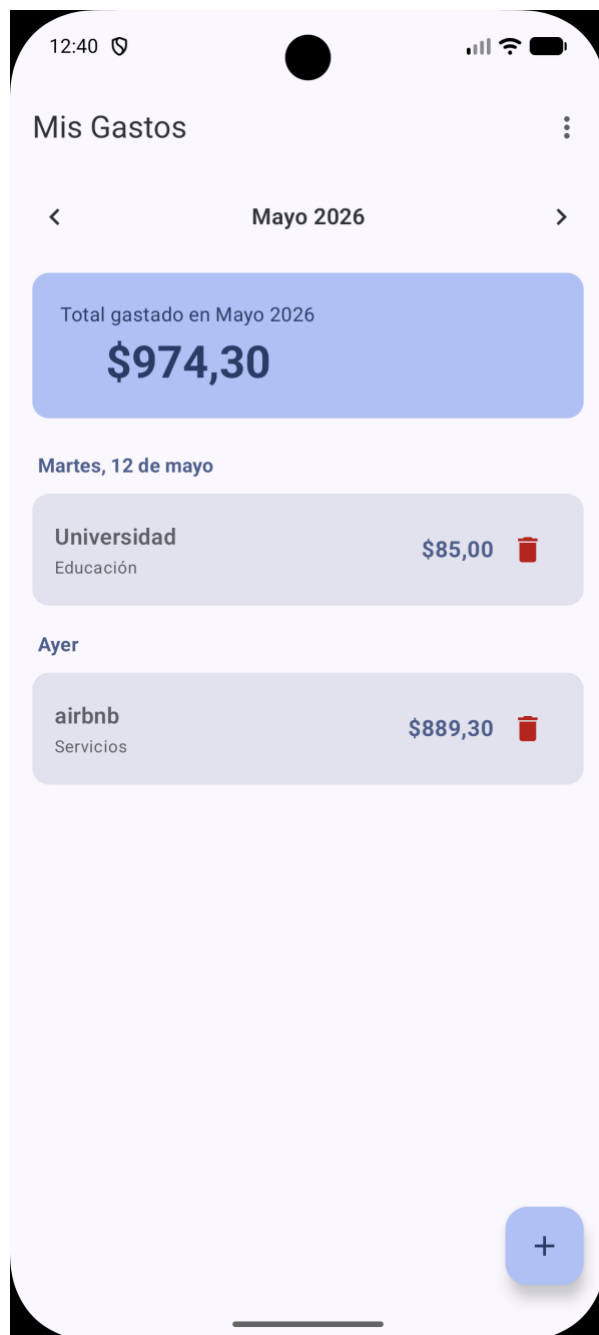


Figura 3. Pantalla principal de gastos. Muestra el navegador de mes, la tarjeta destacada con el total mensual y la lista de gastos agrupados por día con encabezados "Hoy" y "Ayer".

12:40

Cancelar Nuevo gasto

Nombre del gasto

Monto (\$)

Categoría

Comida

Fecha

10 may 2026

Cancelar Guardar

Figura 4. Formulario de creación de gasto. Incluye nombre, monto, selección de categoría mediante chips horizontales y selección de fecha mediante un DatePicker.

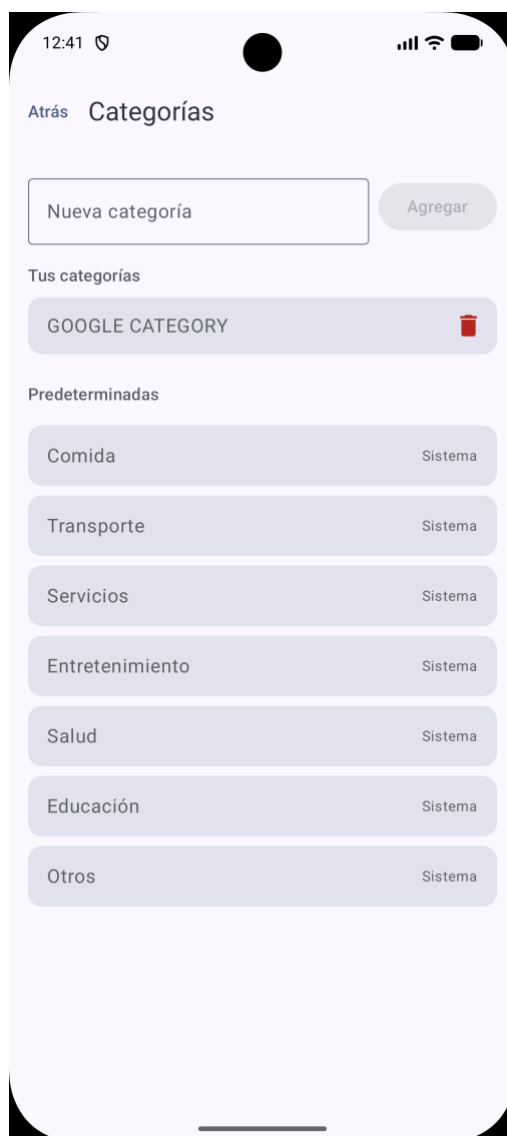


Figura 7. Pantalla de gestión de categorías. Permite agregar y eliminar categorías personalizadas. Las categorías del sistema se muestran como referencia y no son eliminables.

Verificación en la Consola de Firebase

[INSERTAR CAPTURA: Authentication > Users con los usuarios creados]

Figura 8. Pestaña Users de Firebase Authentication. Confirma que los usuarios se crean correctamente en el servicio.

[INSERTAR CAPTURA: Firestore Database con la estructura users/{uid}/gastos]

Figura 9. Estructura de datos en Cloud Firestore. Cada usuario tiene su propia subcolección de gastos y categorías.

Conclusiones

El desarrollo de esta aplicación de control de gastos personales ha permitido aplicar de manera integrada los conocimientos adquiridos durante la unidad sobre desarrollo de aplicaciones móviles en Android con Kotlin. La integración con Firebase Authentication y Cloud Firestore demostró ser una decisión acertada, ya que permitió enfocar el esfuerzo en el dominio del problema (la gestión de gastos) en lugar de en infraestructura accesoria (servidores, bases de datos, mecanismos de autenticación). Esta abstracción es coherente con la tendencia actual de la industria, donde plataformas Backend-as-a-Service liberan a los equipos pequeños y medianos de tareas operativas.

El desarrollo de este proyecto adquirimos conocimientos técnicos que nos resultaron especialmente valiosos. Aprendimos a trabajar con Kotlin como lenguaje principal de Android y a estructurar una aplicación moderna utilizando el patrón Modelo-Vista-VistaModelo (MVVM), separando responsabilidades entre la capa de datos, los ViewModels y las pantallas. Construimos la interfaz con Jetpack Compose, lo que nos permitió comprender cómo funciona un sistema declarativo de UI en contraste con el enfoque tradicional basado en archivos XML. Conocimos cómo se utilizan las corrutinas de Kotlin para ejecutar operaciones asíncronas, así como Flow y StateFlow para mantener la interfaz reactiva al estado de la aplicación. En el lado del backend

Referencias

Firebase. (2024a). Firebase Security Rules: Get Started with Cloud Firestore. Google.

<https://firebase.google.com/docs/firestore/security/get-started>

Firebase. (2024b). Firebase Authentication: Where Do I Start? Google.

<https://firebase.google.com/docs/auth>

Google. (2024). Manage Sign-in for Multiple Identity Providers. Firebase Documentation.

<https://firebase.google.com/docs/auth/android/account-linking>

Google. (2024). Credential Manager: Authenticate Users with Sign in with Google. Android

Developers. <https://developer.android.com/identity/sign-in/credential-manager-siwig>

Google. (2024). Guide to App Architecture. Android Developers.

<https://developer.android.com/topic/architecture>

Google. (2024). Jetpack Compose Documentation. Android Developers.

<https://developer.android.com/jetpack/compose>